



Unidad V

Clase 9

Árboles

ÁRBOLES Y ABB

*Aplicar ABB a problemas concretos y
estimar costo.*

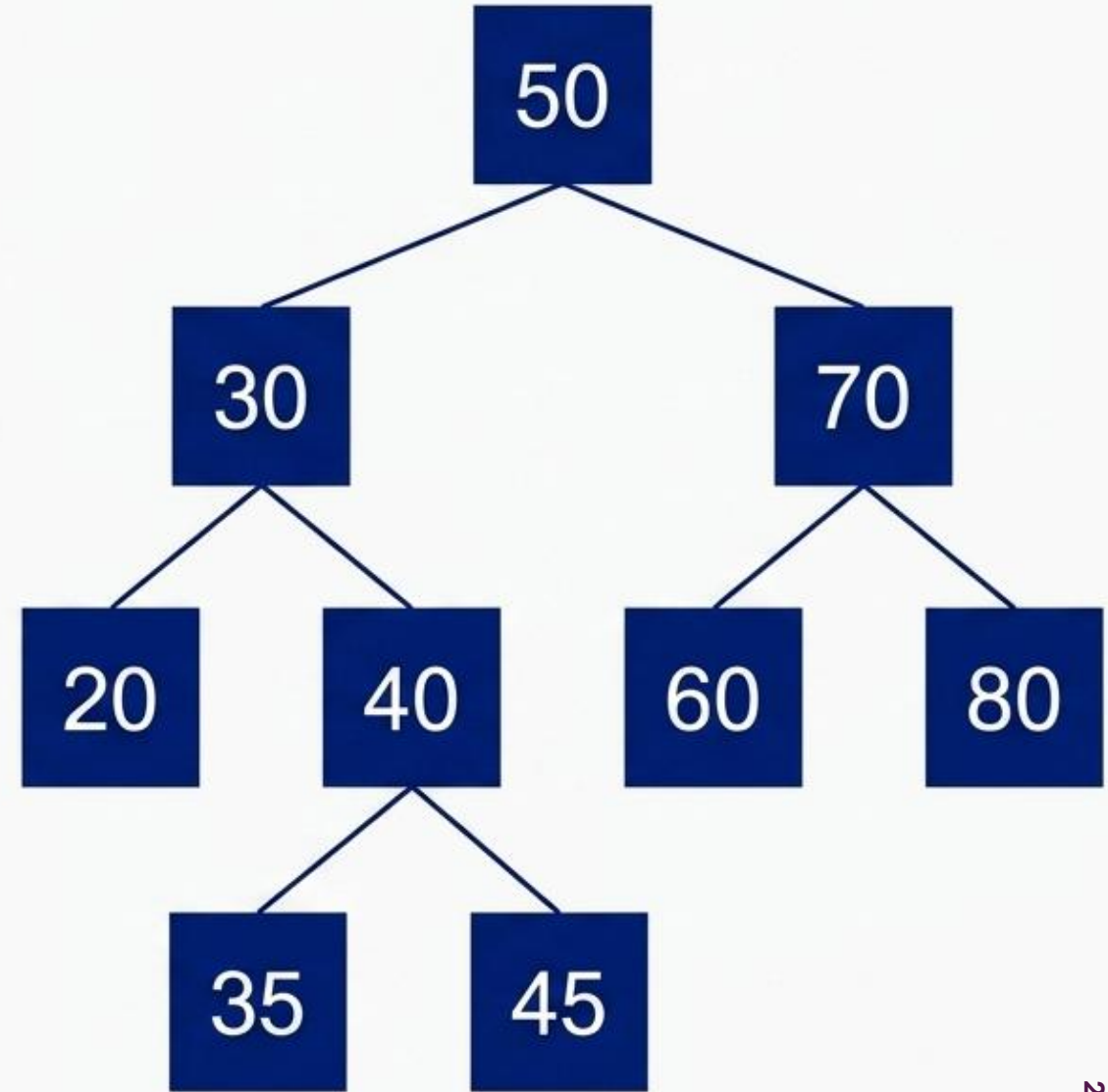
Resolver problemas con ABB

Contexto: En la clase anterior aprendimos a construir e insertar elementos en un Árbol Binario de Búsqueda (ABB), respetando su propiedad de orden.

Objetivo: Transicionar de la construcción mecánica a la resolución de problemas sobre la estructura utilizando recursividad.

Problemas a resolver hoy:

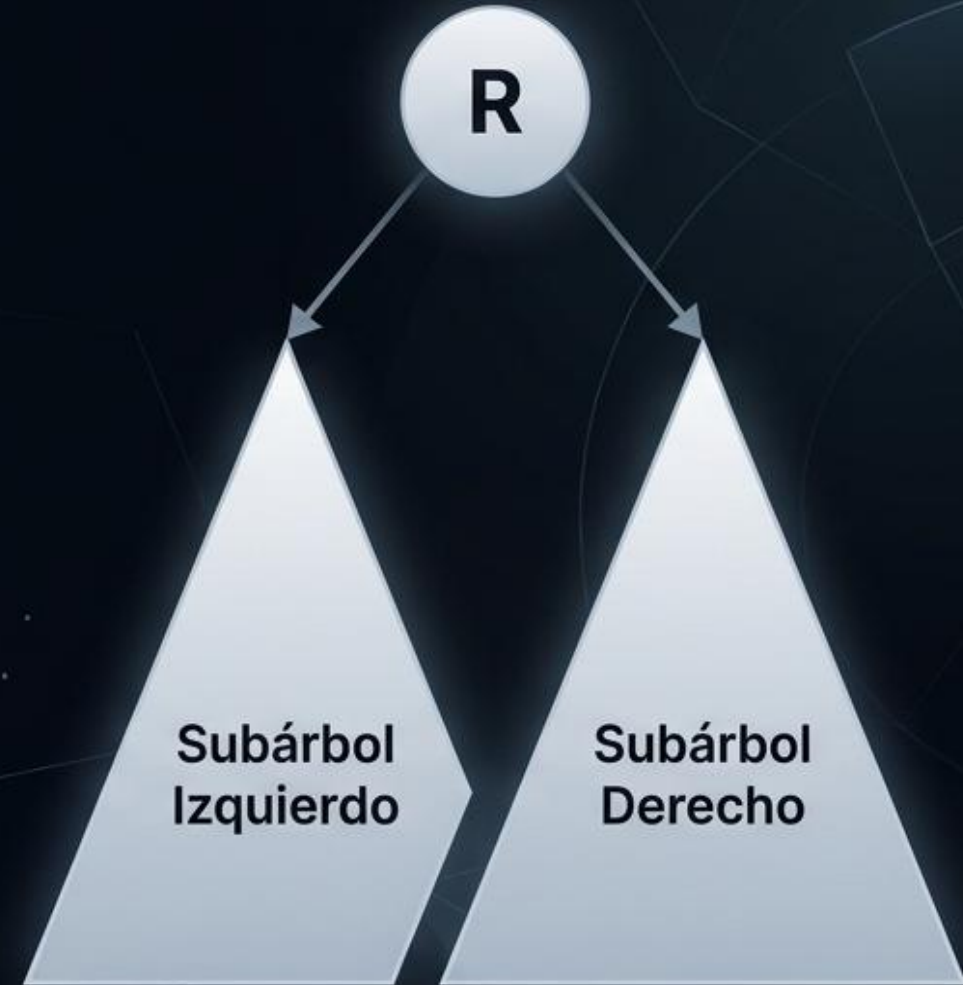
- Contar nodos (Totales, Hojas, Internos)
- Calcular distancias (Altura del árbol)
- Buscar valores (Profundidad de un nodo)
- Analizar el costo computacional (Big O)



El patrón recursivo general

Casi todo problema con ABB se resuelve preguntando tres cosas: qué hacer si está vacío, qué hacer con la raíz, y qué hacer con los hijos.

```
1 public tipo resolver(ABBTDA a) {  
2     if (a.ArbolVacio()) {  
3         // Caso base: ¿Qué devuelvo si no hay árbol?  
4     } else {  
5         // Paso recursivo:  
6         // 1. Proceso la raíz  
7         // 2. resolver(a.HijoIzq())  
8         // 3. resolver(a.HijoDer())  
9     }  
10 }
```



Contar nodos

Lógica: Cada nodo aporta un valor de 1.
El resultado es la suma de la raíz más lo que aporten sus dos subárboles.

```
1 public int contarNodos(ABBTDA a) {  
2     if (a.ArbolVacio()) return 0;  
3     else return 1 + contarNodos(a.HijoIzq())  
4         + contarNodos(a.HijoDer());  
5 }
```



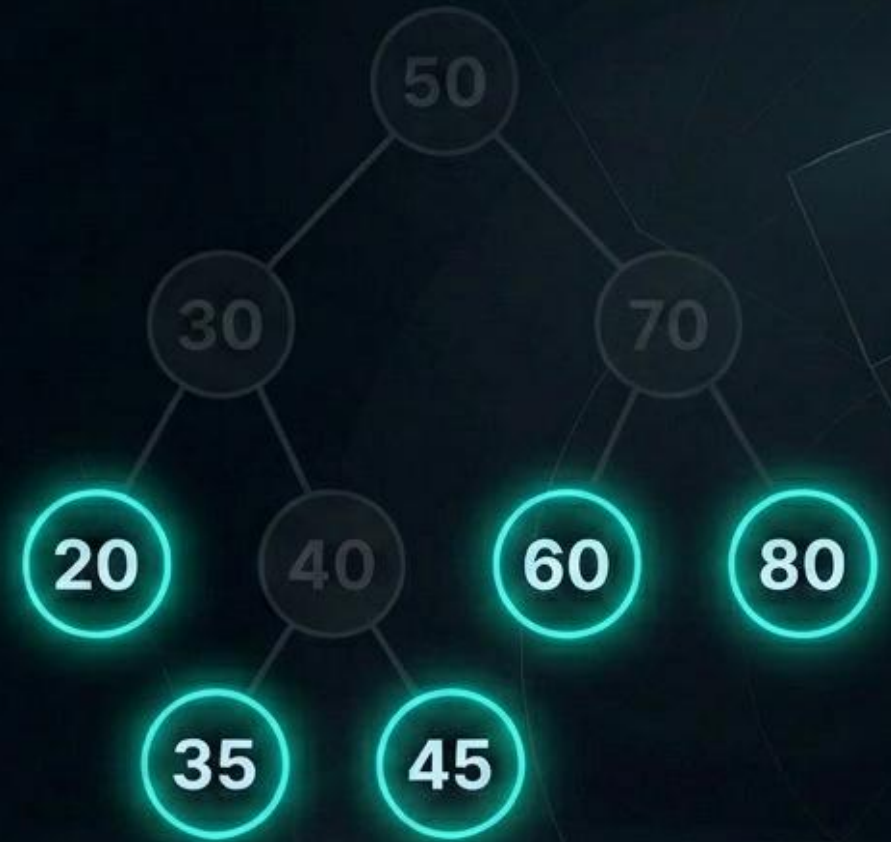
Resultado: 9 nodos

Costo: $O(n)$ (Visita obligatoriamente todos los nodos)

Contar hojas

Definición: Nodo hoja = Nodo sin hijo izquierdo ni derecho.

```
1 public int contarHojas(ABBTDA a) {  
2     if (a.ArbolVacio()) return 0;  
3     else if (a.HijoIzq().ArbolVacio() &&  
4             a.HijoDer().ArbolVacio()) {  
5         return 1; // Es hoja  
6     } else {  
7         return contarHojas(a.HijoIzq()) +  
8             contarHojas(a.HijoDer());  
9     }  
10 }
```



Resultado: 5 hojas (20, 35, 45, 60, 80)

Costo: $O(n)$

Contar nodos internos

Definición: Nodo interno = Nodo con al menos un hijo.

```
1 public int contarInternos(ABBTDA a) {  
2     if (a.ArbolVacio()) return 0;  
3     else if (a.HijoIzq().ArbolVacio() &&  
4             a.HijoDer().ArbolVacio()) {  
5         return 0; // Es hoja, no se cuenta  
6     } else {  
7         return 1 + contarInternos(a.HijoIzq()) +  
8             contarInternos(a.HijoDer());  
9     }  
10 }
```



Resultado: 4 nodos internos (50, 30, 40, 70)

Costo: $O(n)$

Altura del árbol

Definición: Mayor distancia desde la raíz hasta una hoja.

Criterio de la Cátedra:

- Raíz = profundidad 0
- Árbol un solo nodo = altura 0
- Árbol vacío = altura -1

```
1 public int altura(ABBTDA a) {  
2  
3     if (a.ArbolVacio()) return -1;  
4     else return 1 + Math.max(altura(a.HijoIzq()),  
5                             altura(a.HijoDer()));  
6 }
```



Resultado: Altura 3

Costo: $O(n)$

Profundidad de un nodo

Concepto: Explotamos la regla del ABB para bajar solo por el camino correcto, descartando ramas enteras. Se abandona la recursión completa.

```
1 public int profundidad(ABBTDA a, int x) {  
2     If (a.ArbolVacio()) return -1;  
3     else if (a.Raiz() == x) return 0;  
4     else if (x < a.Raiz()) {  
5         int p = profundidad(a.HijoIzq(), x);  
6         if (p == -1) return -1;  
7         else return 1 + p;  
8     } else {  
9         int p = profundidad(a.HijoDer(), x);  
10        if (p == -1) return -1;  
11        else return 1 + p;  
12    }  
13 }
```



Camino: 50 → 30 → 40 → 45

Resultado: Profundidad 3

Costo: $O(h)$

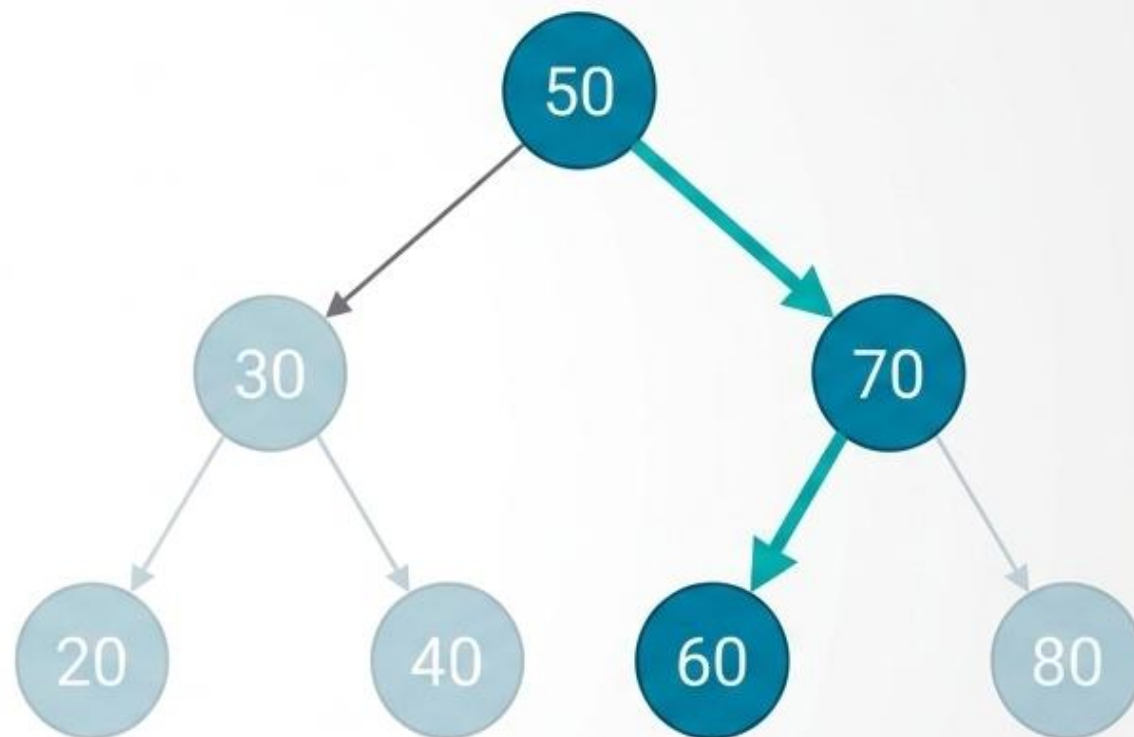
- Balanceado: $O(\log n)$

- Desbalanceado: $O(n)$

Búsqueda dirigida en ABB

La búsqueda aprovecha la regla fundamental del ABB: el **orden**. Al comparar un valor con la **Raiz()**, la estructura nos dirige: menores por el **HijoIzq()**, mayores por el **HijoDer()**. La recursividad emula nuestra toma de decisiones.

```
public boolean existe(ABBTDA a, int x) {  
    if (a.ArbolVacio()) return false;  
    else if (a.Raiz() == x) return true;  
    else if (x < a.Raiz()) return existe(a.HijoIzq(), x);  
    else return existe(a.HijoDer(), x);  
}
```



Menor y mayor valor

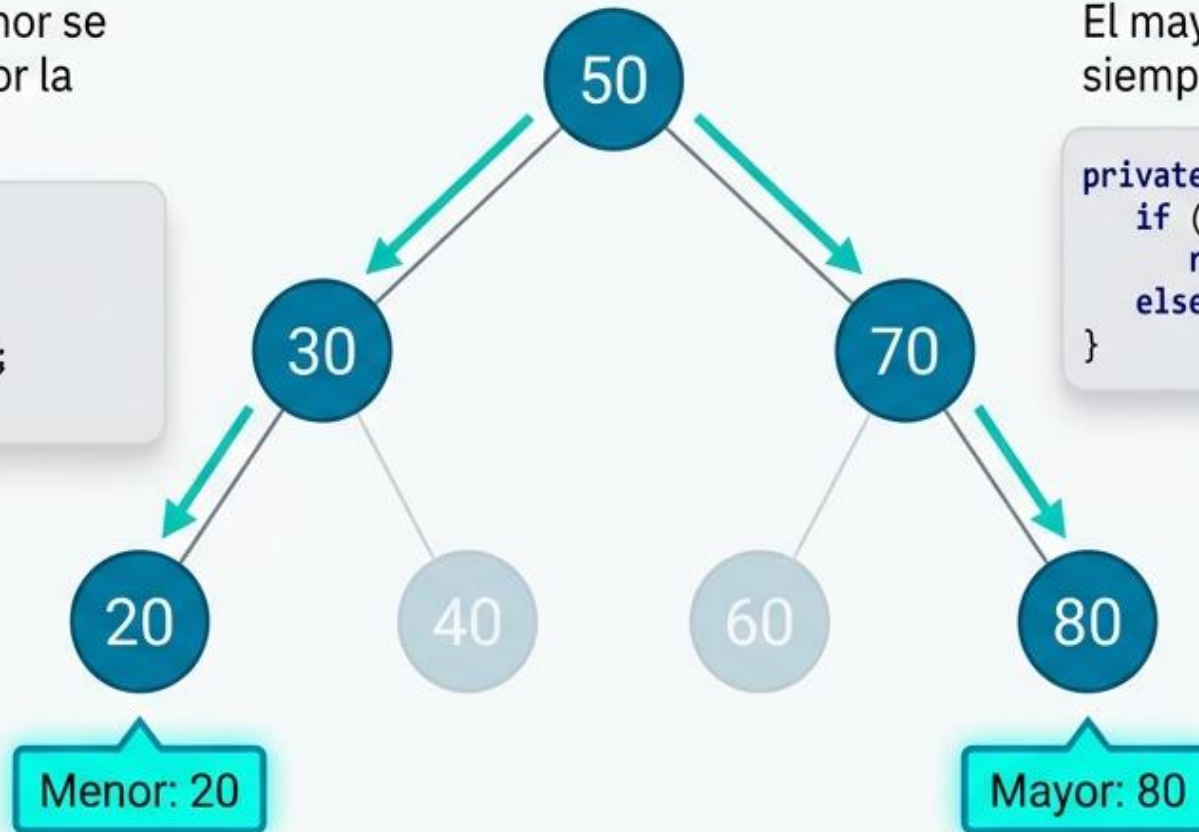
Costo: $O(h)$

Los valores extremos residen en los extremos estructurales. El menor se halla descendiendo siempre por la izquierda.

```
private int menor(ABBTDA a) {  
    if (a.HijoIzq().ArbolVacio())  
        return a.Raiz();  
    else return menor(a.HijoIzq());  
}
```

El mayor valor se halla descendiendo siempre y únicamente por la derecha.

```
private int mayor(ABBTDA a) {  
    if (a.HijoDer().ArbolVacio())  
        return a.Raiz();  
    else return mayor(a.HijoDer());  
}
```

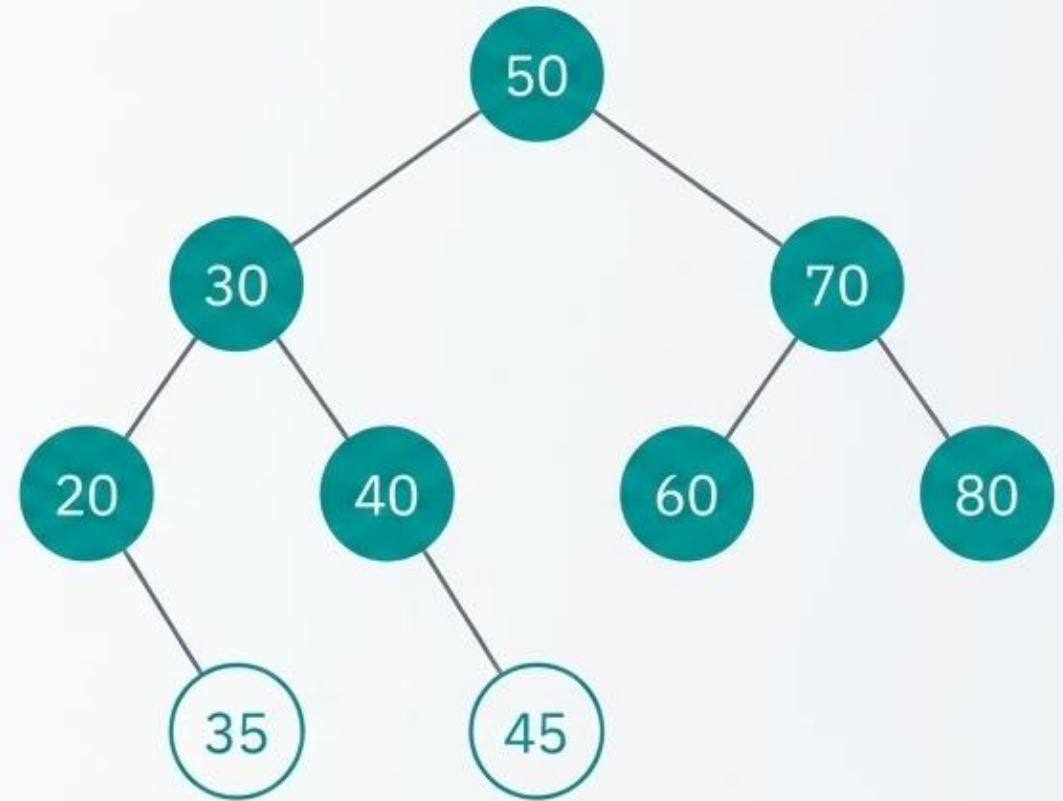


Precondición: El árbol debe estar inicializado y no estar vacío. Llamar a `a.Raiz()` en un `ArbolVacio()` provocaría un error.

Recorrer y filtrar: mostrar pares

No siempre podemos aprovechar la regla de orden. Si buscamos una propiedad matemática (como la paridad), debemos inspeccionar la estructura completa mediante un recorrido.

```
public void mostrarPares(ABBTDA a) {  
    if (!a.ArbolVacio()) {  
        mostrarPares(a.HijoIzq());  
        if (a.Raiz() % 2 == 0)  
            System.out.print(a.Raiz() + " ");  
        mostrarPares(a.HijoDer());  
    }  
}
```



Gracias al recorrido In-Order en un ABB, la salida es inherentemente ordenada.

> Salida In-Order: 20 30 40 50 60 70 80

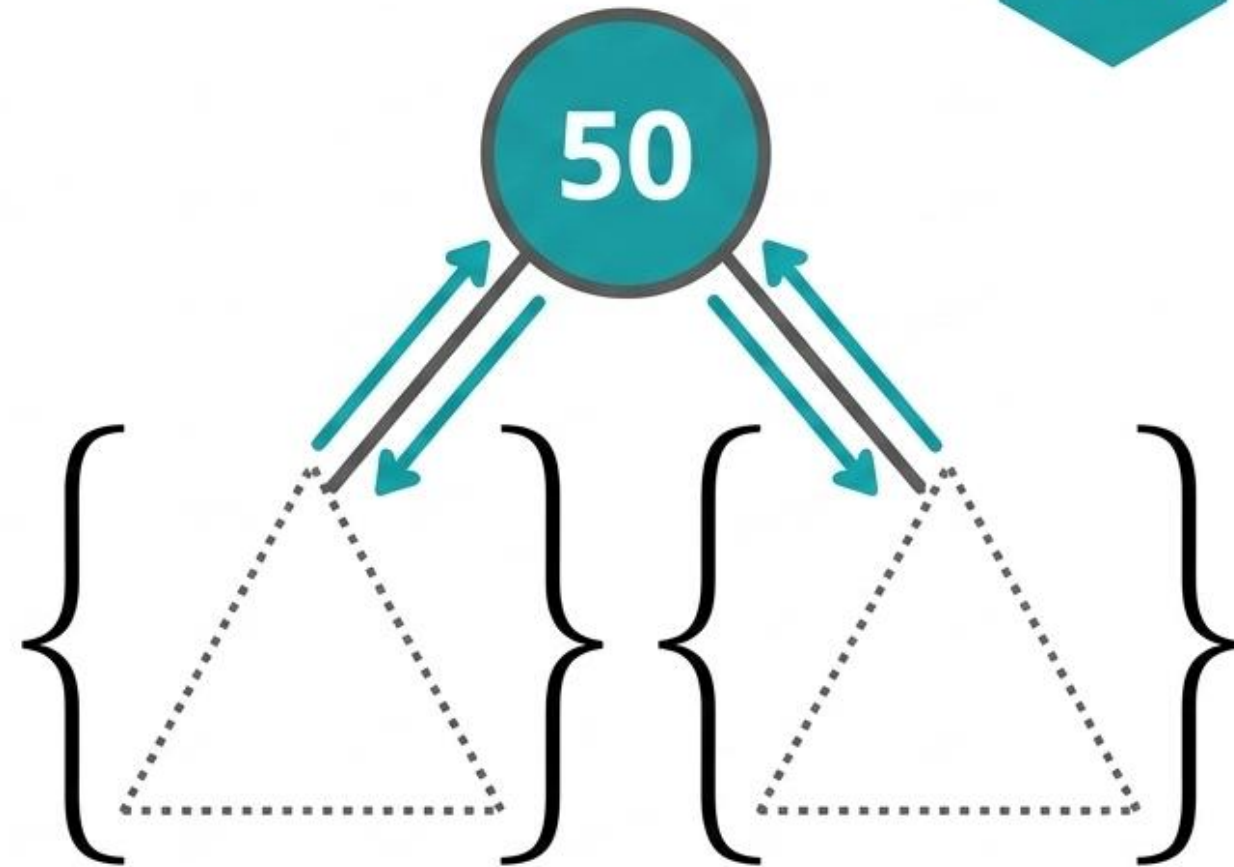
Consolidar resultados: Sumar valores

El ejemplo por excelencia de un recorrido completo.

- **Caso base:** Un árbol vacío aporta 0.
- **Paso recursivo:** El valor de la Raíz + la suma total del subárbol izquierdo + la suma total del subárbol derecho.

```
public int sumar(ABBTDA a) {  
    if (a.ArbolVacio()) return 0;  
    else return a.Raiz() + sumar(a.HijoIzq())  
        + sumar(a.HijoDer());  
}
```

Costo:
 $O(n)$



Suma Izq = 130

Suma Der = 250

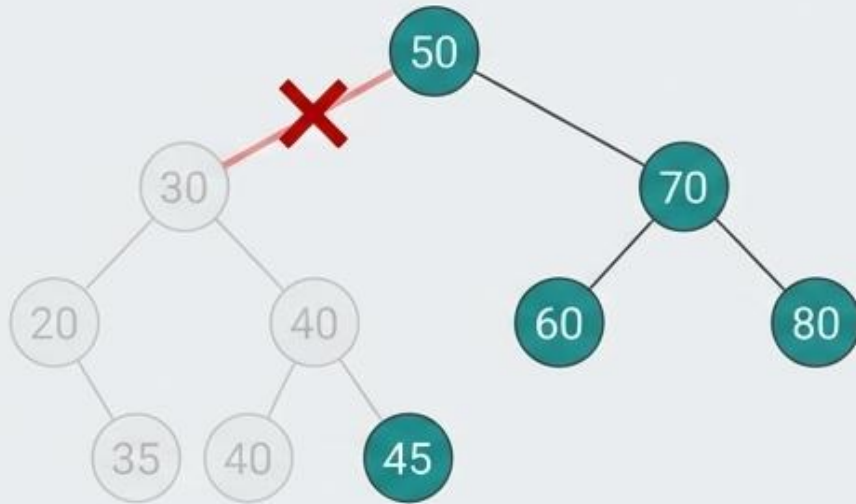
50 (Raíz) + 130 + 250 = **Resultado: 430**

Aprovechar el orden: Poda de ramas

Si buscamos rangos o límites, podemos descartar recursiones completas (podar ramas) que matemáticamente no pueden cumplir la condición, ahorrando cálculos.

Sumar mayores que 45

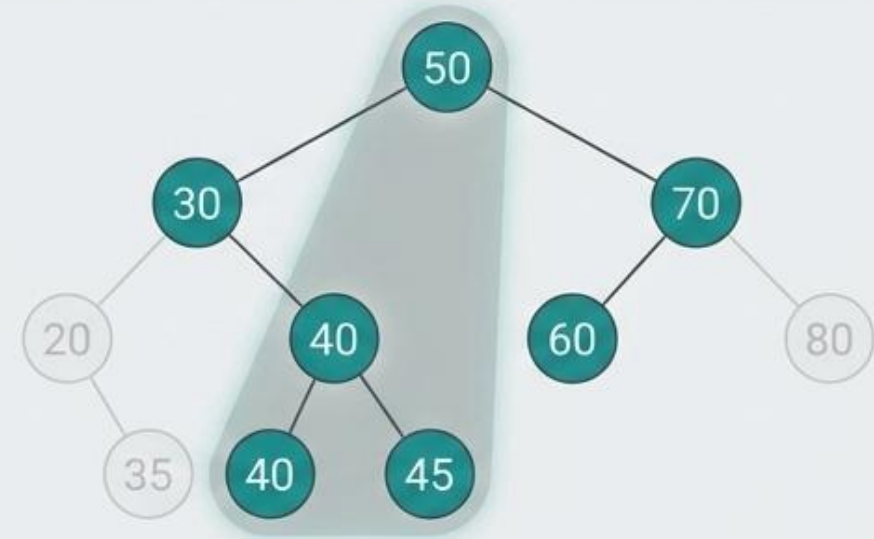
```
if (a.Raiz() <= x)
    return sumarMayoresQue(a.HijoDer(), x);
```



Resultado: 260

Listar rango 35 a 70

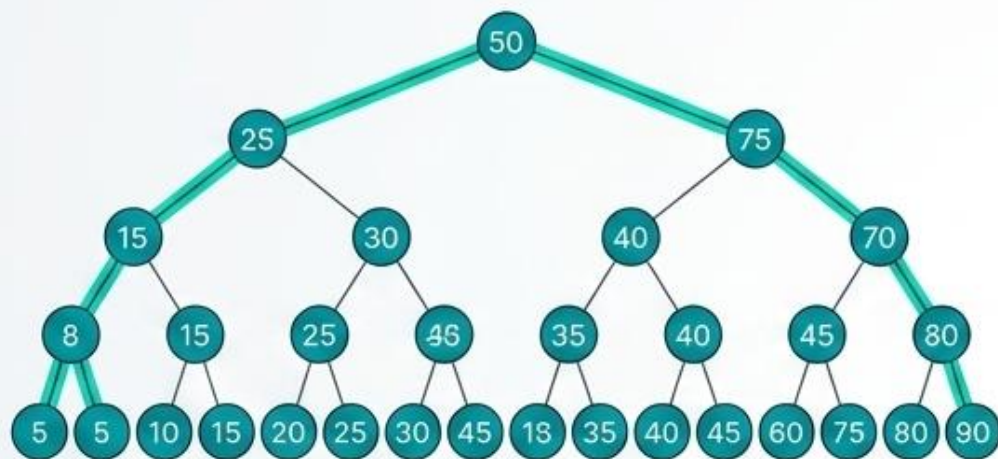
```
if (a.Raiz() > desde) listarRango(Izq);
if (a.Raiz() < hasta) listarRango(Der);
```



Resultado: 35 40 45 50 60 70

El problema oculto: Balance conceptual y costo real

Un árbol puede cumplir perfectamente las reglas lógicas del ABB y, sin embargo, tener un rendimiento degradado. La eficiencia real ($O(h)$) depende estrictamente de su forma física.



✓ ABB Balanceado

Forma óptima. Altura contenida.

$$h \approx \log n$$



✗ ABB Desbalanceado

Forma degradada. Estructura lineal.

$$h \approx n$$

Costo de Búsqueda, Inserción y Eliminación teóricas: $O(h)$

Si el árbol está balanceado $\rightarrow O(\log n)$

✓ (Extremadamente rápido)

Si el árbol es degenerado $\rightarrow O(n)$

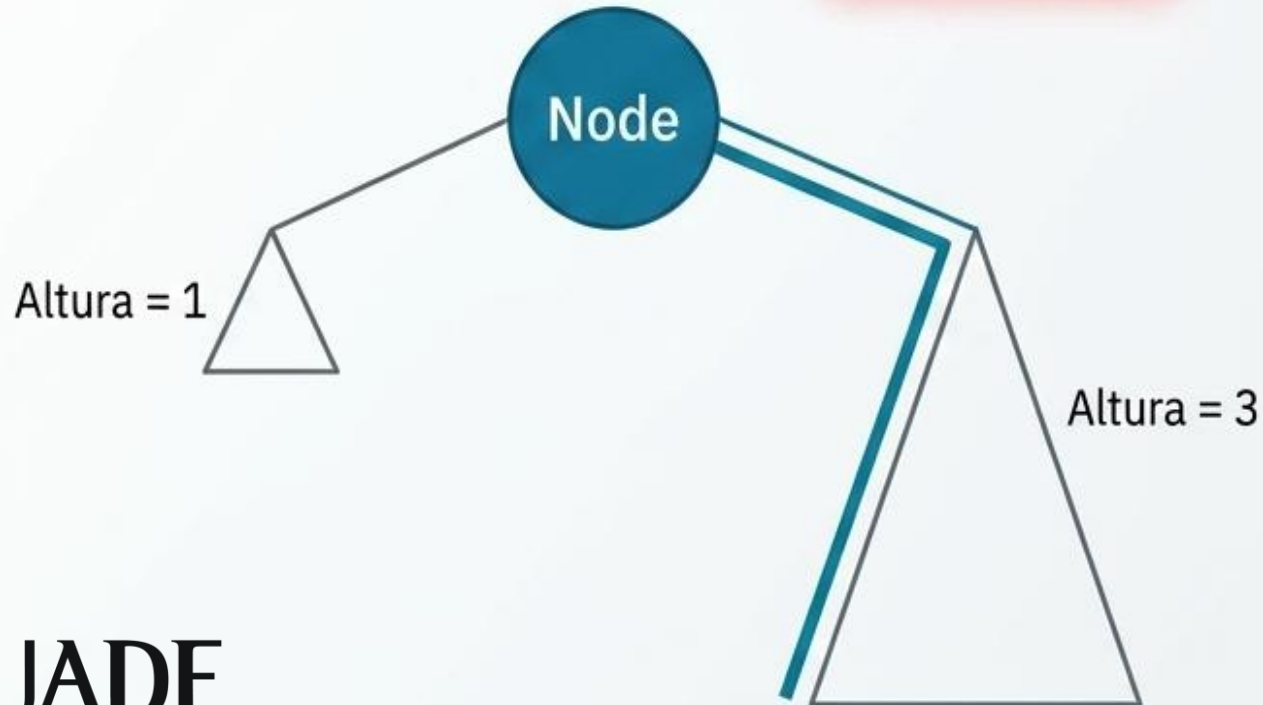
✗ (Se pierde la ventaja del árbol)

Diagnóstico: Detectar nodos desbalanceados

Un nodo se considera desbalanceado si: $| \text{Altura}(\text{Izq}) - \text{Altura}(\text{Der}) | > 1$

$$|1 - 3| = 2$$

⚠ Desbalance



1. altura(a)

Responsabilidad matemática pura (Calcula h).



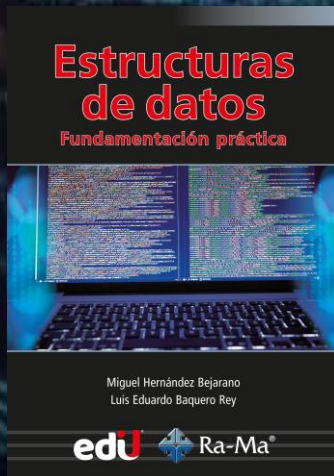
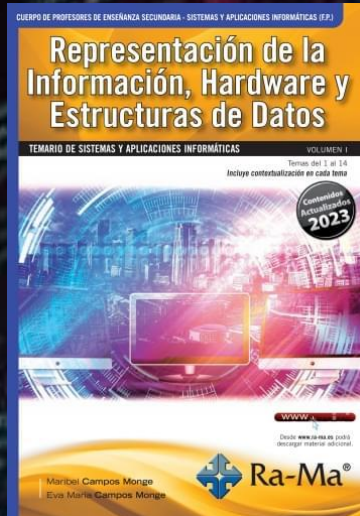
2. estaDesbalanceado(a)

Responsabilidad evaluativa (Calcula la resta absoluta usando altura()).



3. listarDesbalanceados(a)

Responsabilidad operativa (Recorre la estructura aplicando estaDesbalanceado()).



BIBLIOGRAFIA



Cuadrado Estrebou, María Fernanda Trutner, Guillermo Hernán. Revisión: J.Tempio.
Programación II. Apuntes de cátedra.



PROXIMO ENCUENTRO



Clase 10

Cuando el árbol se desbalancea: AVL

Problema del desbalance. Rotaciones simples y dobles.
Garantía $O(\log n)$.